Exploratory Project Report

Change Detection in Satellite Images using U-Net

S.B.Siva Rohith Reddy(24075086)

Harsh Gupta (24075034)

CSE, B.Tech.

Indian Institute of Technology (B.H.U.), Varanasi

Abstract

Change detection in satellite imagery is a fundamental task in remote sensing applications in urban development monitoring, disaster assessment, environmental analysis, and infrastructure planning. The objective of this project is to develop a robust and efficient system for detecting changes between bi-temporal satellite images using deep learning techniques.

In this work, a supervised Convolutional Neural Network (CNN) based on the U-Net architecture is implemented for pixel-wise change detection. The model takes two aligned satellite images captured at different time instances as input and generates a binary change map highlighting the regions where changes have occurred. The LEVIR-CD dataset, consisting of high-resolution aerial images with building-level annotations, is used for training and evaluation.

To efficiently handle large image sizes and improve training performance, the images are resized and divided into smaller patches. The model is trained using a hybrid loss function that combines Binary Cross Entropy and Dice loss, enabling better handling of class imbalance and improving segmentation accuracy.

Additionally, a semi-supervised learning approach based on a Multi-Layer Perceptron with self-training is implemented for comparative analysis. The results demonstrate the effectiveness of deep learning-based methods, particularly U-Net, in capturing spatial patterns and producing accurate change detection maps.

1. Introduction

1.1 Background of Remote Sensing Image Change Detection

Remote sensing has become an essential tool for monitoring the Earth's surface over time. Satellite images captured at different time intervals provide valuable information about changes occurring in urban areas, forests, water bodies, and agricultural land. Change detection refers to the process of identifying differences between such multi-temporal images.

Traditional methods for change detection relied on manual inspection or simple pixel-wise operations such as subtraction or thresholding. However, these methods are highly sensitive to noise, illumination changes, and misalignment between images. With the advancement of deep learning, more sophisticated techniques have been developed that can learn complex spatial patterns and provide more accurate results.

1.2 Scope and Applications

The scope of this project lies in automated detection of changes in satellite imagery, particularly focusing on building-level changes. Such systems are widely used in:

Urban planning and monitoring, where authorities track expansion and unauthorized constructions. Disaster management, where damaged infrastructure can be quickly identified. Environmental monitoring, including deforestation and land-use changes. Agricultural analysis, where crop changes can be observed over time.

1.3 Problem Statement

Given two satellite images of the same geographical area captured at different time instances, the task is to detect and localize the regions where changes have occurred. This problem is challenging due to several factors:

Variations in lighting and seasonal conditions.
Presence of noise and sensor inconsistencies.
Misalignment between images.
Severe class imbalance, where unchanged regions dominate the image.

The goal is to design a model that can accurately produce a pixel-wise binary change map.

1.4 Objectives of the Project

The primary objectives of this project are:

- To design and implement a U-Net based deep learning model for change detection.
- To preprocess large satellite images efficiently using patch-based techniques.
- To generate accurate binary change maps for bi-temporal images.
- To evaluate the performance using standard segmentation metrics.
- To compare the supervised U-Net approach with a semi-supervised learning method.

2. Literature Review

2.1 Traditional Methods for Change Detection

Early approaches to change detection relied heavily on pixel-based operations such as image differencing, image ratioing, and thresholding. These methods are computationally simple but often fail to capture complex spatial relationships. They are highly sensitive to noise and variations in illumination, making them unreliable for real-world applications.

2.2 Semi-Supervised Learning for Change Detection (MLP with KNN-based Label Propagation)

The semi-supervised method implemented in this project is based on a Multi-Layer Perceptron (MLP) combined with a self-training strategy. Unlike fully supervised learning, this approach utilizes both labeled and unlabeled data.

The input images are first converted to grayscale and normalized. For each pixel, a **3×3 neighborhood patch** is extracted from both images, and the flattened patches are concatenated to form a feature vector. This results in a representation that captures local spatial context.

The dataset is divided into labeled and unlabeled subsets in a balanced manner to address class imbalance. Initially, the MLP model is trained on the labeled data. The trained model is then used to predict labels for unlabeled data. Only predictions with high confidence (greater than a threshold of 0.95) are selected and added to the labeled dataset.

This iterative process continues, gradually increasing the size of the labeled dataset. To maintain class balance, equal numbers of samples from both classes are selected during each iteration. The model architecture consists of two hidden layers with 128 and 64 neurons, respectively, and uses early stopping to prevent overfitting.

Although this method reduces the need for labeled data, it is limited in capturing global spatial features and often produces noisy predictions compared to CNN-based models.

2.3 CNN-Based Architectures for Pixel-wise Prediction

Convolutional Neural Networks have revolutionized image processing tasks by enabling automatic feature extraction. Unlike traditional methods, CNNs can learn hierarchical representations of data, capturing both local and global spatial information.

In change detection, CNNs process bi-temporal images and produce pixel-wise predictions. This makes them highly suitable for segmentation tasks where precise localization is required.

2.4 U-Net Architecture

U-Net is a fully convolutional neural network originally designed for biomedical image segmentation. It consists of two main parts: a contracting path (encoder) and an expanding path (decoder). Skip connections are used to transfer information from the encoder to the decoder, preserving spatial details.

2.4.1 Encoder (Contracting Path)

The encoder consists of repeated convolutional blocks followed by max pooling operations. Each block contains two convolutional layers with batch normalization and ReLU activation. As the network goes deeper, the spatial dimensions decrease while the number of feature channels increases.

2.4.2 Decoder (Expanding Path)

The decoder performs upsampling using transposed convolution. The upsampled features are concatenated with corresponding encoder features using skip connections. This helps in recovering spatial details lost during downsampling.

2.4.3 Skip Connections

Skip connections play a crucial role in U-Net by preserving fine-grained information. They allow the decoder to access high-resolution features from the encoder, resulting in more accurate segmentation.

3. Dataset Description

3.1 Overview of Dataset

The LEVIR-CD dataset is used in this project. It consists of high-resolution satellite images with building-level change annotations.

3.2 Data Organization

The dataset is structured into training, validation, and testing sets. Each split contains three folders:

- A: Image before change
- B: Image after change
- Label: Ground truth change mask

3.3 Image Characteristics

The original images have a resolution of **1024 × 1024 pixels** and are provided in RGB format. The ground truth masks are binary images indicating changed and unchanged regions.

3.4 Patch Extraction Strategy

Due to the large size of the images, they are resized to **256 × 256** and divided into **four 128 × 128 patches**. This reduces computational cost and increases the number of training samples.

4. Data Preprocessing

4.1 Normalization and Scaling

All images are converted to tensors with values in the range [0,1]. Ground truth labels are binarized using a threshold operation.

4.2 Patch Generation

Each resized image is divided into a grid of 2×2 patches. Corresponding patches from images A, B, and label are extracted to ensure spatial alignment.

4.3 Input Preparation

The two images are concatenated along the channel dimension, resulting in a **6-channel input tensor**. The corresponding label patch is a single-channel binary mask.

5. Proposed Method: Supervised CNN-Based U-Net Model

5.1 Problem Formulation

The objective of this work is to perform **pixel-wise binary classification** for change detection. Given two aligned satellite images, denoted as Image A and Image B, captured at different time instances, the task is to predict a change map Y, where each pixel indicates whether a change has occurred.

The problem can be mathematically expressed as:

$$\mathbf{f(A, B) \rightarrow Y}$$

where:

- A and B are RGB images of size $\mathbf{3 \times H \times W}$
- Y is a binary mask of size $\mathbf{1 \times H \times W}$

Each pixel in the output mask is defined as:

- $Y(i, j) = 1 \rightarrow$ change detected
- $Y(i, j) = 0 \rightarrow$ no change

5.2 Input Representation (Bi-temporal Image Pair)

The two input images are concatenated along the channel dimension to form a single input tensor.

Since each image has 3 channels (RGB), the combined input becomes:

Input size = 6 × H × W

For patch-based training, each input patch is of size:

6 × 128 × 128

This representation allows the model to jointly learn spatial and temporal differences between the two images.

5.3 Model Configuration

5.3.1 Network Architecture Overview

The model used in this project is a U-Net architecture implemented using PyTorch. It consists of:

- 4 Encoder blocks
- 1 Bottleneck layer
- 4 Decoder blocks
- Final output layer with sigmoid activation

The model contains approximately **31 million trainable parameters**, enabling it to capture complex spatial features.

5.3.2 Encoder (Contracting Path)

Each encoder block consists of:

- Two convolutional layers (kernel size = 3, padding = 1)

- Batch Normalization
- ReLU activation
- Max pooling (2 × 2)

At each level:

- Spatial dimensions are reduced by half
- Feature channels are increased

This helps in extracting high-level semantic features.

5.3.3 Bottleneck Layer

The bottleneck is the deepest part of the network where:

- Feature maps have the lowest spatial resolution
- Feature channels are highest

It captures global contextual information before the decoding process begins.

5.3.4 Decoder (Expanding Path)

Each decoder block performs:

- Upsampling using transposed convolution
- Concatenation with encoder features (skip connection)
- Two convolutional layers with BatchNorm and ReLU

This restores spatial resolution while preserving important details.

5.3.5 Skip Connections

Skip connections link encoder and decoder layers at corresponding levels. They help in:

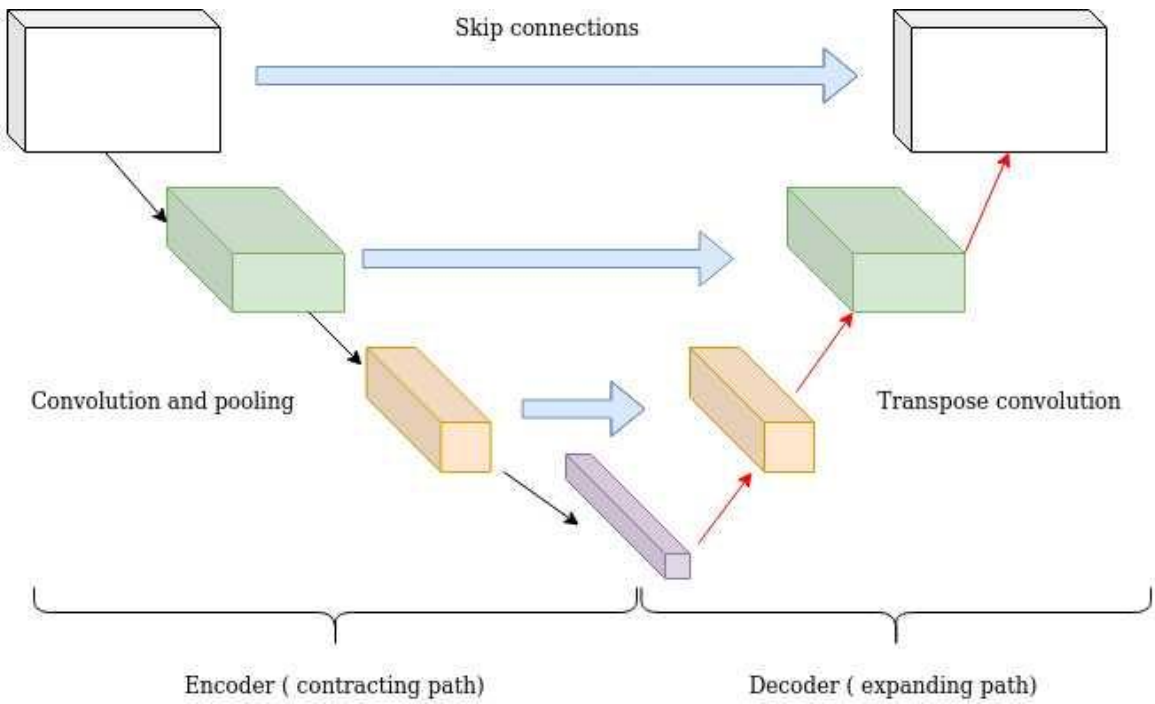
- Preserving spatial information
- Improving gradient flow
- Enhancing segmentation accuracy

5.3.6 Output Layer

The final layer consists of:

- A 1 × 1 convolution
- Sigmoid activation

This produces a probability map with values between 0 and 1.



5.4 Loss Function

A hybrid loss function is used:

Loss = 0.5 × BCE + 0.5 × Dice Loss

5.4.1 Binary Cross Entropy (BCE)

BCE measures pixel-wise classification error:

$$\mathbf{BCE = -\left[\, y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \,\right]}$$

5.4.2 Dice Loss

Dice coefficient measures overlap between prediction and ground truth:

$\text{Dice} = (2 \times |P \cap G|) / (|P| + |G|)$

Dice loss is defined as:

$\text{Dice Loss} = 1 - \text{Dice}$

Reason for Combination

- BCE ensures pixel-level accuracy
- Dice improves overlap between regions
- Combined loss helps handle class imbalance

5.5 Training Strategy

5.5.1 Supervised Learning Setup

The model is trained using labeled data from the dataset. Each training sample consists of:

- Input: 6-channel image patch
- Target: binary mask

5.5.2 Optimization Algorithm

The model is trained using the **Adam optimizer**, which adapts learning rates for efficient convergence.

Learning rate:

$\alpha = 0.0001$

5.5.3 Hyperparameters

- Epochs: 15
- Batch size: 4
- Patch size: 128 × 128
- Threshold (inference): 0.4

5.5.4 Training Behavior

During training:

- Training loss decreases steadily
- Validation IoU improves over epochs
- Best model is saved based on highest IoU

The model achieves the best validation IoU of approximately **0.7257**.

```
 Epoch │  Train Loss │  Val Loss │  Val IoU │  Val F1 │   Time
──────────────────────────────────────────────────────────────────
     1 │     0.61006 │   0.53598 │   0.4017 │  0.5731 │ 171.4s
         ✓ Best model saved (IoU = 0.4017)
     2 │     0.44975 │   0.41158 │   0.5543 │  0.7133 │ 167.4s
         ✓ Best model saved (IoU = 0.5543)
     3 │     0.34477 │   0.35131 │   0.6130 │  0.7600 │ 168.6s
         ✓ Best model saved (IoU = 0.6130)
     4 │     0.27458 │   0.30378 │   0.6605 │  0.7955 │ 166.4s
         ✓ Best model saved (IoU = 0.6605)
     5 │     0.23002 │   0.28976 │   0.6243 │  0.7687 │ 167.0s
     6 │     0.19643 │   0.25846 │   0.6861 │  0.8138 │ 163.3s
         ✓ Best model saved (IoU = 0.6861)
     7 │     0.18319 │   0.27225 │   0.6274 │  0.7711 │ 167.2s
     8 │     0.17467 │   0.25153 │   0.6784 │  0.8084 │ 163.6s
     9 │     0.16307 │   0.24589 │   0.6754 │  0.8063 │ 162.8s
    10 │     0.15651 │   0.22420 │   0.7224 │  0.8388 │ 164.6s
         ✓ Best model saved (IoU = 0.7224)
    11 │     0.14571 │   0.22391 │   0.7254 │  0.8409 │ 166.2s
         ✓ Best model saved (IoU = 0.7254)
    12 │     0.14369 │   0.22019 │   0.7210 │  0.8379 │ 166.2s
    13 │     0.14933 │   0.22971 │   0.7257 │  0.8410 │ 164.5s
         ✓ Best model saved (IoU = 0.7257)
    14 │     0.13933 │   0.22845 │   0.7166 │  0.8349 │ 164.9s
    15 │     0.12565 │   0.23525 │   0.6910 │  0.8173 │ 162.5s

[INFO] Training complete. Best Val IoU: 0.7257
```

5.6 Validation Strategy

Validation is performed after each epoch using the validation dataset.

Metrics used:

- IoU (Intersection over Union)
- F1 Score

The model with the highest IoU is selected.

5.7 Output Generation (Change Detection Mask)

The model outputs a probability map which is converted into a binary mask using thresholding:

- If prediction > 0.4 → change
- Otherwise → no change

6. Experimental Setup

6.1 Implementation Environment

The proposed change detection system is implemented using the Python programming language with the PyTorch deep learning framework. The experiments are conducted in the Google Colab environment, which provides access to GPU acceleration, enabling efficient training of deep neural networks.

The implementation utilizes several libraries including NumPy for numerical operations, PIL for image processing, and torchvision for image transformations. The use of GPU significantly reduces training time, especially given the large number of model parameters (approximately 31 million) and the size of the dataset.

The device configuration is dynamically determined in the code using:

- CUDA (GPU) if available
- Otherwise CPU

This ensures flexibility and portability of the implementation across different hardware environments.

6.2 Dataset Splitting and Patch Generation

The dataset used in this project follows a predefined split into training, validation, and testing sets. Each split contains three subdirectories corresponding to:

- Image A (before image)
- Image B (after image)
- Ground truth label (binary change mask)

Due to the large resolution of the original images (1024 × 1024), direct training on full images is computationally expensive. To address this, a patch-based strategy is employed.

Each image is first resized to **256 × 256** pixels. Subsequently, it is divided into **four non-overlapping patches** of size **128 × 128** using a grid-based approach. This results in a significant increase in the number of training samples and improves memory efficiency.

The total number of patches used in the experiment is:

- Training set: **1780 patches**
- Validation set: **256 patches**
- Test set: **512 patches**

This patch-based approach allows the model to learn local spatial features more effectively while maintaining computational feasibility.

6.3 Training Configuration (U-Net Model)

The U-Net model is trained using a supervised learning approach. Each input sample consists of a 6-channel tensor formed by concatenating the corresponding patches from Image A and Image B. The target output is a single-channel binary mask indicating the changed regions.

The training process is carried out using mini-batch gradient descent with the following configuration:

- Number of epochs: 15

- Batch size: 4
- Learning rate: 0.0001
- Optimizer: Adam

The Adam optimizer is chosen due to its adaptive learning rate mechanism, which helps in faster convergence and stable training. The loss function used is a combination of Binary Cross Entropy and Dice loss, which balances pixel-wise accuracy and region overlap.

During each epoch, the model processes all training patches, computes the loss, and updates the weights using backpropagation. The training loss is monitored to ensure proper convergence of the model.

6.4 Evaluation Metrics

To assess the performance of the model, several standard evaluation metrics are used. These metrics are computed by comparing the predicted binary mask with the ground truth mask.

Let:

- TP = True Positives
- FP = False Positives
- FN = False Negatives

Precision

Precision measures the proportion of correctly predicted change pixels among all predicted change pixels:

Precision = TP / (TP + FP)

Recall

Recall measures the proportion of correctly detected change pixels among all actual change pixels:

Recall = TP / (TP + FN)

F1 Score

The F1 score is the harmonic mean of precision and recall:

F1 Score = (2 × Precision × Recall) / (Precision + Recall)

Intersection over Union (IoU)

IoU measures the overlap between predicted and ground truth regions:

IoU = TP / (TP + FP + FN)

These metrics provide a comprehensive evaluation of both detection accuracy and segmentation quality.

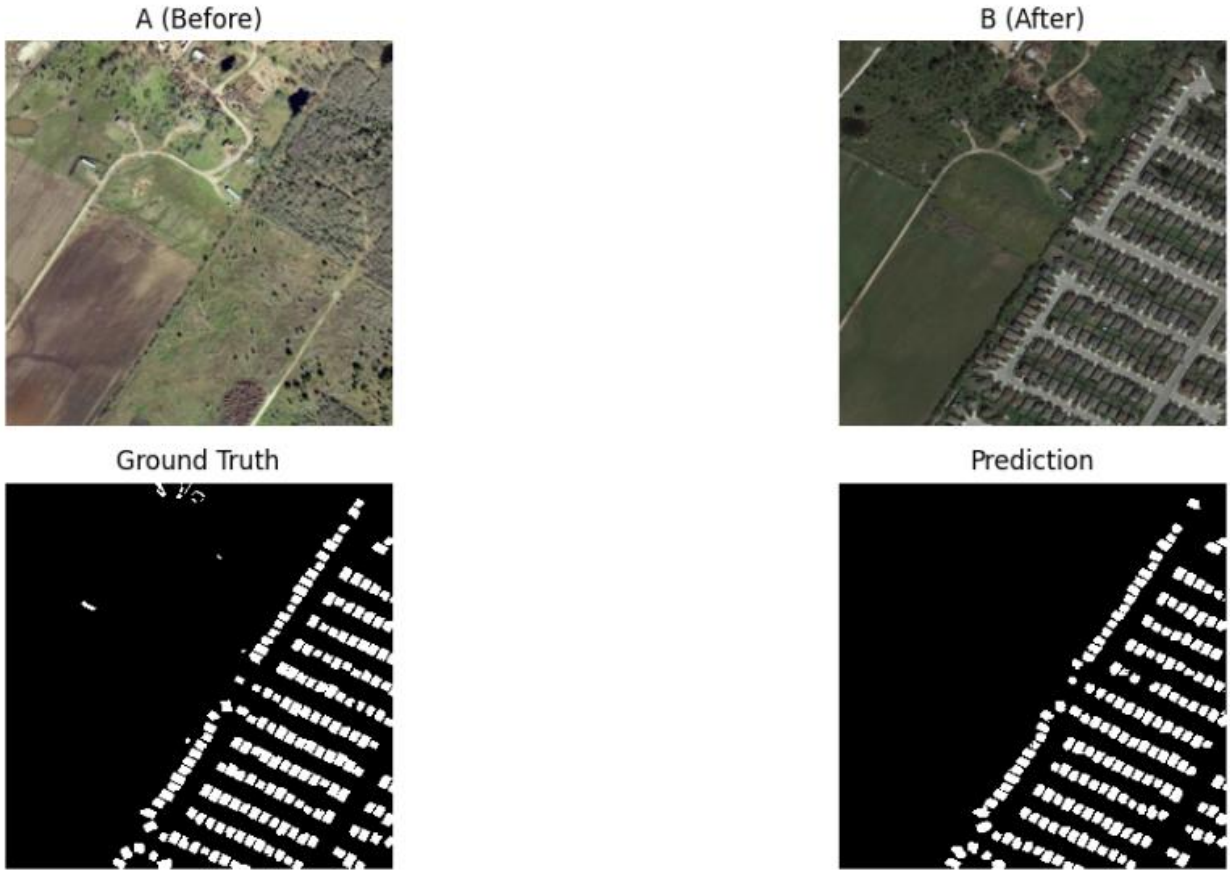
6.5 Testing Procedure

After training, the best-performing model is selected based on the highest validation IoU and saved. This model is then used for testing on the unseen test dataset.

During testing, the following steps are performed:

1. The saved model is loaded into memory.
2. Each test image is resized to 256 × 256.
3. The image is divided into four patches of size 128 × 128.
4. Each patch is passed through the trained U-Net model to obtain a prediction.
5. The predicted patches are recombined to reconstruct the full-size prediction map.
6. A threshold of 0.4 is applied to convert the probability map into a binary mask.

This patch-wise inference ensures that high-resolution images can be processed efficiently without exceeding memory constraints.



7. Challenges

7.1 Image Alignment and Noise Issues

Satellite images may have slight misalignment due to sensor and acquisition differences. This can lead to false detections. Additionally, noise, illumination changes, and seasonal variations make it difficult to distinguish real changes from irrelevant variations.

7.2 Limited Labeled Data

Deep learning models require large labeled datasets, which are often limited. Although patch-based training increases the number of samples, it does not fully replace the need for diverse labeled data.

7.3 Class Imbalance

Most pixels belong to the “no change” class, leading to class imbalance. This can bias the model, which is addressed using a combined BCE and Dice loss.

7.4 High Computational Cost

Training U-Net is computationally expensive due to its large number of parameters and high-resolution inputs. Patch-based training and GPU usage help manage this.

8. Results and Comparison

8.1 Results of Proposed U-Net Model

The U-Net model achieved the following results on the test dataset:

- Precision: 0.8626
- Recall: 0.8173
- F1 Score: 0.8393
- IoU: 0.7231

These results indicate accurate detection with a good balance between precision and recall.

8.2 Results of Semi-Supervised MLP-Based Method

The MLP-based semi-supervised method shows lower performance due to its inability to capture spatial features effectively and its sensitivity to noise.

8.3 Quantitative Comparison

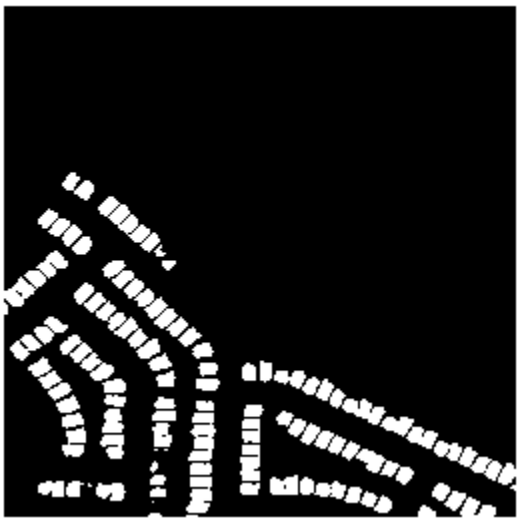
U-Net outperforms the MLP method across all metrics due to its ability to learn spatial features and use skip connections for better segmentation.

8.4 Qualitative Comparison

The results show that the U-Net model accurately identifies most of the changed regions, with predictions closely matching the ground truth. The model produces clear boundaries with minimal noise, although small errors may occur in some regions.

Ground Truth

U-Net Prediction



SSL MLP Prediction

9. Conclusion and Future Work

9.1 Summary of Work

A U-Net based model was developed for change detection using bi-temporal satellite images. Patch-based training and a combined loss function were used to improve performance. A semi-supervised MLP method was also implemented for comparison.

9.2 Key Findings

- U-Net achieves high accuracy in change detection
- BCE + Dice loss improves segmentation
- Patch-based training is effective
- CNN-based methods outperform MLP

9.3 Future Enhancements

In real-world scenarios, only limited ground truth data is available (e.g., after natural disasters). The trained U-Net model can be used as a base, and semi-supervised learning can be applied using the small labeled data to predict changes in the remaining regions.